

Using printf for Aligned Output

A quick guide and reference

A word before you read. These are my notes from over the years, cleaned up so they are readable. A few things to keep in mind:

- **They are not a study guide, and they are not gospel.** Not everything here appears in a lecture, and not every topic has notes yet; I will write more as we go.
- **I write from memory and revise every semester.** A typo or an imprecise statement can survive longer than it should.
- **I use AI to help** convert documents to markdown, check for issues, and add explanations. I do not trust it completely, so I am still editing these myself.
- **Java changes.** The basics we cover are pretty resilient, but some features do move over time.

So: if you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. When in doubt, the textbook is the tiebreaker, and the compiler beats us both.

This is a reference, not an assignment. It exists to show you the mechanics of `printf` so you can apply them to your own fields, in whatever program calls for them. The worked example below uses a small inventory list, not adventurers or ability scores, on purpose: the technique is exactly what you need, but the specifics are yours to work out.

1. Why printf

Building output with `+` works, but it gives you no control over spacing:

```
System.out.println("Widget" + " " + "Hardware" + " " + 12 + " " + 4.50);  
System.out.println("Big Bracket" + " " + "Hardware" + " " + 3 + " " + 12.0);
```

```
Widget Hardware 12 4.5  
Big Bracket Hardware 3 12.0
```

Nothing lines up, and the two prices even print with a different number of decimal places. `printf` solves both problems: it takes a format string with placeholders, followed by the values to fill them in, and gives you direct control over width, alignment, and decimal places.

2. Basic Syntax

```
System.out.printf("%s is %d years old.%n", "Grom", 34);
```

Each % placeholder in the format string is filled in, in order, by the arguments that follow it. %s takes a String, %d takes an integer, and %n inserts a newline.

printf is not just for System.out. A PrintWriter has a printf method that works exactly the same way, except it writes to whatever file the PrintWriter was opened on instead of the console:

```
PrintWriter writer = new PrintWriter("report.txt");
writer.printf("%s is %d years old.%n", "Grom", 34);
writer.close();
```

3. Quick Reference

Table 1: Common printf specifiers

Specifier	Meaning	Example
%d	integer	printf("%d", 8)
%5d	integer, right-aligned, width 5	printf("%5d", 8)
%-5d	integer, left-aligned, width 5	printf("%-5d", 8)
%s	String	printf("%s", "Grom")
%-10s	String, left-aligned, width 10	printf("%-10s", "Grom")
%f	floating point, 6 decimals default	printf("%f", 3.14159)
%.2f	floating point, 2 decimal places	printf("%.2f", 3.14159)
%6.2f	floating point, width 6, 2 decimals	printf("%6.2f", 3.14159)
%n	newline (platform independent)	
%%	a literal percent sign	printf("100%%")

A width without a minus sign right-aligns; a width with a minus sign left-aligns. As a rule of thumb, left-align text, right-align numbers, so that a column of numbers stays lined up on its ones digit no matter how many digits each one has.

4. A Worked Example

Start from the same three inventory rows, with no formatting at all:

```
System.out.printf("%s %s %d %f%n", "Widget", "Hardware", 12, 4.50);
System.out.printf("%s %s %d %f%n", "Big Bracket", "Hardware", 3, 12.0);
```

```
Widget Hardware 12 4.500000
Big Bracket Hardware 3 12.000000
```

This is already better: both prices now show the same number of decimal places. But %f defaults to six decimal places, more than a price needs, and nothing is aligned yet. Add widths, alignment, and a decimal limit:

```
System.out.printf("%-12s %-9s %4d  $%6.2f%n", "Widget", "Hardware", 12, 4.50);
System.out.printf("%-12s %-9s %4d  $%6.2f%n", "Big Bracket", "Hardware", 3, 12.0);
System.out.printf("%-12s %-9s %4d  $%6.2f%n", "Screw", "Fasteners", 500, 0.05);
```

Widget	Hardware	12	\$	4.50
Big Bracket	Hardware	3	\$	12.00
Screw	Fasteners	500	\$	0.05

Reading the format string piece by piece: `%-12s` is the item name, left-aligned in a 12-character-wide column, so short names like "Widget" do not leave the next column ragged. `%-9s` does the same for the category. `%4d` right-aligns the quantity in a 4-character column, so 12, 3, and 500 all line up on their last digit. A literal `$` is written directly into the format string, and `%6.2f` right-aligns the price in a 6-character column with exactly two decimal places.

5. Common Mistakes

- **Using `\n` instead of `%n`.** `\n` works on most machines, but `%n` is the version that is guaranteed to produce the correct newline on whatever system the program actually runs on. Use `%n` in a `printf` format string.
- **Mismatched types.** `%d` expects an integer type. Passing a `double` where `%d` expects an `int` throws an exception at runtime:

```
double price = 4.50;  
System.out.printf("%d%n", price);
```

```
Exception in thread "main" java.util.IllegalFormatConversionException:  
d != java.lang.Double
```

Use `%f` (or `%.2f`) for a `double`, and `%d` only for an `int` or `long`.

- **Forgetting to close a `PrintWriter`.** Output written with `PrintWriter` is not guaranteed to actually reach the file until `close()` is called. A program that writes but never closes may produce an empty or incomplete file.
- **Too few or too many arguments.** Every `%` placeholder in the format string needs a matching argument, in the same order. A mismatch either throws an exception or silently prints the wrong value in the wrong place.